

Министерство образования Республики Беларусь

Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра программного обеспечения информационных технологий

Дисциплина
«Объектно-ориентированные технологии программирования и
стандарты проектирования»

ОТЧЁТ
к лабораторной работе №3

СЕРИАЛИЗАЦИЯ И ДЕСЕРИАЛИЗАЦИЯ ДАННЫХ
В РАМКАХ ПРОЕКТА «NINJA-ARENA»

Студент группы
№351001
Ушаков А.Д.

Преподаватель
Деменковец Д. В.

Минск 2025

СОДЕРЖАНИЕ

Введение	3
1 Теоретические сведения	4
1.1 JSON и глубокое копирование объектов	4
1.2 Методы JSON	5
1.3 Сериализация и десериализация.....	6
2 Реализация.....	7
2.1 Модернизация доступа к локальному хранилищу браузера	7
2.2 Парсинг JSON-файла с иерархией классов	9
2.3 Сериализация и десериализация объектов	9
2.4 Представление данных в файлах	14
2.5 UI (container_page.html)	15
3 Тестирование	16
3.1 Открытие диалогового окна.....	16
3.2 Загрузка контейнера на страницу	16
3.3 Скачивание контейнера	17
Заключение	19
Список использованных источников	20

ВВЕДЕНИЕ

В рамках данной лабораторной работы продолжена разработка проекта "Ninja-Arena", направленного на создание системы для динамического управления иерархией объектов ниндзя. Основной акцент в текущей работе сделан на реализацию механизмов сериализации и десериализации данных, что позволяет сохранять и восстанавливать состояние объектов в различных форматах, включая JSON и TXT. Это расширяет функциональность системы, обеспечивая гибкость в работе с данными и их переносимость между сеансами работы приложения.

Ключевой задачей стало обеспечение корректного преобразования сложных объектов иерархии ниндзя в текстовые форматы и обратно, с сохранением всех свойств и структуры данных. Реализованные механизмы позволяют не только сохранять данные в локальном хранилище браузера, но и экспортировать их в файлы, а также загружать из внешних источников. Это особенно важно для пользователей, которым необходимо передавать данные между устройствами или создавать резервные копии.

Особенностью реализации является поддержка как структурированного формата JSON, так и простого текстового формата, что делает систему доступной для различных сценариев использования. Например, JSON обеспечивает удобство машинной обработки, а TXT — простоту чтения и редактирования вручную. Кроме того, система включает валидацию данных, что гарантирует их целостность при загрузке и предотвращает ошибки, связанные с некорректными форматами или несоответствием структуре классов.

Реализованные механизмы сериализации и десериализации тесно интегрированы с существующей системой динамического создания объектов, что позволяет сохранять и восстанавливать даже сложные иерархии ниндзя с их уникальными свойствами. Это демонстрирует эффективное сочетание объектно-ориентированного подхода с современными веб-технологиями, обеспечивая надёжность и удобство работы с данными.

В результате проделанной работы система приобрела новые возможности для управления данными, что делает её более универсальной и удобной для пользователей. Реализованные функции сериализации и десериализации являются важным шагом в развитии проекта, открывая перспективы для дальнейшего расширения функционала, такого как интеграция с облачными хранилищами или поддержка дополнительных форматов данных.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1 JSON и глубокое копирование объектов

Материал о формате JSON и глубоком копировании через JSON представлен на рисунке 1.1.1.

 **1. JSON (JavaScript Object Notation)**

JSON — это текстовый формат обмена данными, основанный на синтаксисе объектов JavaScript. Используется для хранения и передачи структурированной информации в виде пар **ключ–значение**.

Основные правила:

- Ключи — **всегда** в двойных кавычках: "key"
- Значения могут быть:
 - строки: "value"
 - числа: 123
 - булевы: true, false
 - массивы: []
 - объекты: {}
 - null

Применение:

- Обмен данными между клиентом и сервером (API)
- Хранение состояния приложения (localStorage)
- Конфигурационные файлы (например, classes.json)

 **2. Глубокая копия объекта через JSON**

 **Проблема:**
Обычное копирование (`const copy = obj`) создаёт ссылку, а не новый объект.

 **Решение:**

```
const original = { a: 1, nested: { b: 2 } };
const deepCopy = JSON.parse(JSON.stringify(original));

deepCopy.nested.b = 100;
console.log(original.nested.b); // 2 (не изменилось)
```

 **Плюсы:**

- Полностью независимая копия

 **Минусы:**

- Не работает с:
 - функциями
 - Date
 - undefined
 - циклическими ссылками

Рисунок 1.1.1 – JSON и глубокое копирование

1.2 Методы JSON

Среди методов JSON в JavaScript (TypeScript) основополагающими являются `stringify()` и `parse()`, смотреть рисунок 1.2.1.

✂ 3. Методы JSON

◆ `JSON.stringify(obj)`

Преобразует объект в строку JSON (сериализация):

```
const ninja = { name: "Naruto", rank: "S", skills: ["Rasengan", "Shadow Clone"] };
const jsonString = JSON.stringify(ninja);
console.log(jsonString);
// '{"name":"Naruto","rank":"S","skills":["Rasengan","Shadow Clone"]}'
```

✦ **Использование:**

- Сохранение в `localStorage`:
`localStorageAccessor.serializeContainer()`
- Отправка данных на сервер

◆ `JSON.parse(jsonString)`

Преобразует строку JSON обратно в объект (десериализация):

```
const restoredNinja = JSON.parse(jsonString);
console.log(restoredNinja.name); // "Naruto"
```

✦ **Использование:**

- Загрузка данных из хранилища:
`localStorageAccessor.deserializeContainer()`

Рисунок 1.2.1 – Методы JSON

1.3 Сериализация и десериализация

Информация о сериализации и десериализации [1], а также особенностях их реализации в проекте представлена на рисунке 1.3.1.



4. Сериализация и десериализация

- **Сериализация** — преобразование объекта в строку для хранения или передачи:
`localStorage.setItem('data', JSON.stringify(obj));`
 - ♦ В проекте: `prepareNinjaData()` подготавливает объект к сериализации.
- **Десериализация** — восстановление объекта из строки:
`const obj = JSON.parse(localStorage.getItem('data'));`
 - ♦ В проекте: `createDynamicNinja()` создаёт объект ниндзя из данных JSON.



5. Особенности реализации в проекте



Динамическая десериализация

- Классы ниндзя создаются через фабрику (`createDynamicNinja`), используя поле `type` в JSON.



Валидация данных

- Проверка:
 - структуры: `validateNinjaObject`
 - допустимого типа: `validateNinjaType`



Поддержка формата TXT

- Альтернативный формат для ручного редактирования: `parseTxtContent`



Итог

Реализованные механизмы обеспечивают:

- Гибкое управление данными
- Сохранность между сеансами
- Совместимость с внешними форматами (JSON/TXT)
- Возможность безопасной сериализации и восстановления объектов ниндзя

Рисунок 1.3.1 – Сериализация и десериализация

2 РЕАЛИЗАЦИЯ

2.1 Модернизация доступа к локальному хранилищу браузера

Данные об объектах располагаются в локальном хранилище браузера [2]. Для создания механизма сериализации через файлы необходимо было задействовать часть кода из `LocalStorageAccessor.js` в других файлах, поэтому пришлось прибегнуть к изменениям в самом `LocalStorageAccessor.js` и выделению функции подготовки данных в отдельный файл `prepare_data.js`.

Файл `prepare_data.js`:

```
export function prepareNinjaData(ninja, classData) {
  const baseData = {
    name: ninja.name,
    health: ninja.health,
    chakra: ninja.chakra,
    rank: ninja.rank,
    organization: ninja.organization,
    village: ninja.village,
    appearance: ninja.appearance,
    arenaView: ninja.arenaView,
  };

  const classInfo = classData[ninja.constructor.name];

  if (classInfo?.fields) {
    classInfo.fields.forEach(field => {
      baseData[field.name] = ninja[field.name];
    });
  }

  return {
    type: ninja.constructor.name,
    data: baseData,
  };
}
```

Обновлённый LocalStorageAccessor.js:

```
import { prepareNinjaData } from "../ninjaData/prepare_data.js";

export class LocalStorageAccessor {

  static serializeContainer(container, classData) {
    const serializedContainer = container.map( ninja =>
prepareNinjaData(ninja, classData) );

    console.log('Serialized Container:', serializedContainer);
    localStorage.setItem('Container',
JSON.stringify(serializedContainer));
  }

  static async deserializeContainer(container) {
    let result;
    let {createDynamicNinja} = await
import('../build/models/factories/ninjaFactory.js');
    const savedContainerData =
JSON.parse(localStorage.getItem('Container'));

    console.log('Restored Container:', savedContainerData);

    if (savedContainerData) {
      result = savedContainerData.map( ({ type, data }) => { return
createDynamicNinja(type, data) } );
    } else {
      result = container;
    }

    return result;
  }
}
```


2.2 Парсинг JSON-файла с иерархией классов

Для того, чтобы получить информацию о существующей иерархии классов, нужно обработать файл `classes.json` функцией, которая определена в файле `processNinjaData.js`:

```
export async function createHierarchy(pathToJSON) {  
  let result;  
  
  const responseData = await fetch(pathToJSON);  
  result = await responseData.json();  
  
  return result.classes;  
}
```

2.3 Сериализация и десериализация объектов

Файл `check_data.js` с вспомогательными функциями:

```
function isJsonFile(file) {  
  return file.name.endsWith('.json');  
}  
  
function isJsonContent(content) {  
  const trimmedContent = content.trim();  
  
  return trimmedContent.startsWith('[') &&  
    trimmedContent.endsWith(']');  
}  
  
export function shouldParseAsJson(file, content) {  
  return isJsonFile(file) || isJsonContent(content);  
}  
  
export function validateNinjaObject(item) {  
  
  if (!item.type || !item.data) {  
    throw new Error("Invalid ninja object structure");  
  }  
  
  return item;  
}  
  
export function validateNinjaType(item, classData) {  
  
  if (!classData[item.type]) {  
    throw new Error(`Unknown ninja type: ${item.type}`);  
  }  
}
```

```

    return item;
}

export function ensureFileExtension(fileName, extension) {
    return fileName.endsWith(extension) ? fileName :
`${fileName}${extension}`;
}

```

А также в prepare_data_for_file.js:

```

import { prepareNinjaData } from "../../ninjaData/prepare_data.js";

export function prepareForFile(container, classData) {
    return container.map( ninja => prepareNinjaData(ninja, classData) );
}

```

Обработка файлов в parse_data.js:

```

import { createHierarchy } from "../../ninjaData/processNinjaData.js";
import { shouldParseAsJson, validateNinjaObject, validateNinjaType }
from "../check_data.js";

function parseTxtContent(content) {
    const lines = content.split('\n');
    const result = [];
    let currentNinja = null;

    for (const line of lines) {

        if (line.startsWith('Type: ')) {

            if (currentNinja) {
                result.push(currentNinja);
            }

            currentNinja = { type: line.substring(6).trim(), data: {} };

        } else if (currentNinja && line.includes(': ')) {
            const [key, value] = line.split(': ').map(s => s.trim());

            currentNinja.data[key.toLowerCase()] = isNaN(value) ? value :
Number(value);
        }

    }

    if (currentNinja) {
        result.push(currentNinja);
    }
}

```

```

    return result;
}

async function parseJsonContent(content) {
    const parsedData = JSON.parse(content);
    if (!Array.isArray(parsedData)) {
        throw new Error("Invalid JSON format: expected array of ninjas");
    }
    return parsedData;
}

async function validateData(data, classData) {

    return data.map(item => {
        const validatedItem = validateNinjaObject(item);

        return validateNinjaType(validatedItem, classData);
    });
}

export async function parseFileContent(file, content) {
    let parsedData;

    if ( shouldParseAsJson(file, content) ) {
        parsedData = await parseJsonContent(content);
    } else {
        parsedData = parseTxtContent(content);
    }

    const classData = await
createHierarchy('../ninjaData/classes.json');

    return validateData(parsedData, classData);
}

```

Чтение и скачивание файлов в `up_download_data.js`:

```

import { ensureFileExtension } from "../check_data.js";

export function readFileAsText(file) {

    return new Promise((resolve, reject) => {
        const reader = new FileReader();

        reader.onload = (event) => resolve(event.target.result);
        reader.onerror = () => reject(new Error("Error reading file"));

        reader.readAsText(file);
    });
}

```

```

}

export function downloadSerializedData(data, fileName) {
  const jsonData = JSON.stringify(data, null, 2);

  createDownloadLink(jsonData, ensureFileExtension(fileName, '.json'),
    'application/json');

  let txtData = '';

  data.forEach(item => {
    txtData += `Type: ${item.type}\n`;
    Object.entries(item.data).forEach(([key, value]) => {
      txtData += `${key}: ${value}\n`;
    });
    txtData += '\n';
  });

  createDownloadLink(txtData, ensureFileExtension(fileName, '.txt'),
    'text/plain');
}

function createDownloadLink(content, fileName, mimeType) {
  const blob = new Blob([content], { type: mimeType });
  const url = URL.createObjectURL(blob);

  const link = document.createElement('a');
  link.href = url;
  link.download = fileName;
  document.body.appendChild(link);
  link.click();
  document.body.removeChild(link);
  URL.revokeObjectURL(url);
}

```

Основной файл file_de_serialize.js:

```

import { createHierarchy } from "../ninjaData/processNinjaData.js";
import { prepareForFile } from
  "../file_operations/prepare_data_for_file.js";
import { parseFileContent } from "../file_operations/parse_data.js";
import { readFileAsText, downloadSerializedData } from
  "../file_operations/up_download_data.js";
import { LocalStorageAccessor } from "../localStorageAccessor.js";

export async function serialize(container, options = {}) {
  const { format = 'localStorage', fileName = null } = options;
  const classData = await
  createHierarchy('../ninjaData/classes.json');
  let result;

```

```

    if (format === 'localStorage') {
      LocalStorageAccessor.serializeContainer(container, classData);
      result = true;
    } else if (format === 'file') {
      result = prepareForFile(container, classData);

      if (fileName) {
        downloadSerializedData(result, fileName);
      }
    } else {
      throw new Error(`Unknown serialization format: ${format}`);
    }
  }

  return result;
}

export async function deserialize(options = {}) {
  const { source = 'localStorage', file = null } = options;
  let result;

  if (source === 'localStorage') {
    result = await LocalStorageAccessor.deserializeContainer();
  } else if (source === 'file') {
    if (!file) {
      throw new Error('File is required for file deserialization');
    }

    const fileContent = await readFileAsText(file);

    result = await parseFileContent(file, fileContent);
  } else {
    throw new Error(`Unknown deserialization source: ${source}`);
  }

  return result;
}

export async function uploadContainer() {
  return new Promise((resolve, reject) => {
    const input = document.createElement('input');
    input.type = 'file';
    input.accept = '.json,.txt';

    input.onChange = async (e) => {

```

```

    const file = e.target.files[0];

    if (!file) {
      return reject(new Error('No file selected'));
    }

    try {
      const containerData = await deserialize({ source: 'file', file
    });

    localStorage.setItem('Container',
JSON.stringify(containerData));
    resolve(true);

    } catch (error) {
      reject(error);
    }
  };

  input.click();
});
}

export async function downloadCurrentContainer() {
  const container = await deserialize();

  await serialize(container, { format: 'file', fileName:
'ninja_container' });

  return true;
}

```

2.4 Представление данных в файлах

Представление данных в JSON-файле можно увидеть на рисунке 2.4.1.

```

{
  "type": "sensorNinja",
  "data": {
    "name": "Hinata",
    "health": 800,
    "chakra": 400,
    "rank": 1,
    "organization": "none",
    "village": "Konohagakure",
    "appearance": "../../../src/components/characters/hinata/images/hinata_ava.jpg"
  },
  {
    "type": "clairvoyant",
    "data": {
      "name": "Itachi",
      "health": 1200,
      "chakra": 600,
      "rank": 3,
      "organization": "akatsuki",
      "village": "Konohagakure",
      "appearance": "../../../src/components/characters/itachi/images/itachi_ava.jpg"
    }
  }
}

```

Рисунок 2.4.1 – Данные в JSON

Обязательное форматирование для текстового файла выглядит следующим образом (рис. 2.4.1):

```
Type: sensorNinja
name: Hinata
health: 800
chakra: 400
rank: 1
organization: none
village: Konohagakure
appearance: ../../../../src/components/characters/hinata/images/hinata_ava.jpg
arenaView: undefined

Type: clairvoyant
name: Itachi
health: 1200
chakra: 600
rank: 3
organization: akatsuki
village: Konohagakure
appearance: ../../../../src/components/characters/itachi/images/itachi_ava.jpg
arenaView: undefined

Type: birdwatchers
name: Jiraiya
health: 1400
chakra: 800
rank: 4
organization: sanins
village: Konohagakure
appearance: ../../../../src/components/characters/jiraiya/images/jiraiya_ava.jpg
arenaView: undefined
```

Рисунок 2.4.1 – Данные в TXT

2.5 UI (container_page.html)

На страницу container_page.html было добавлено 2 кнопки для загрузки контейнера и скачивания (если локальное хранилище не пустое) – смотреть рисунок 2.5.1.

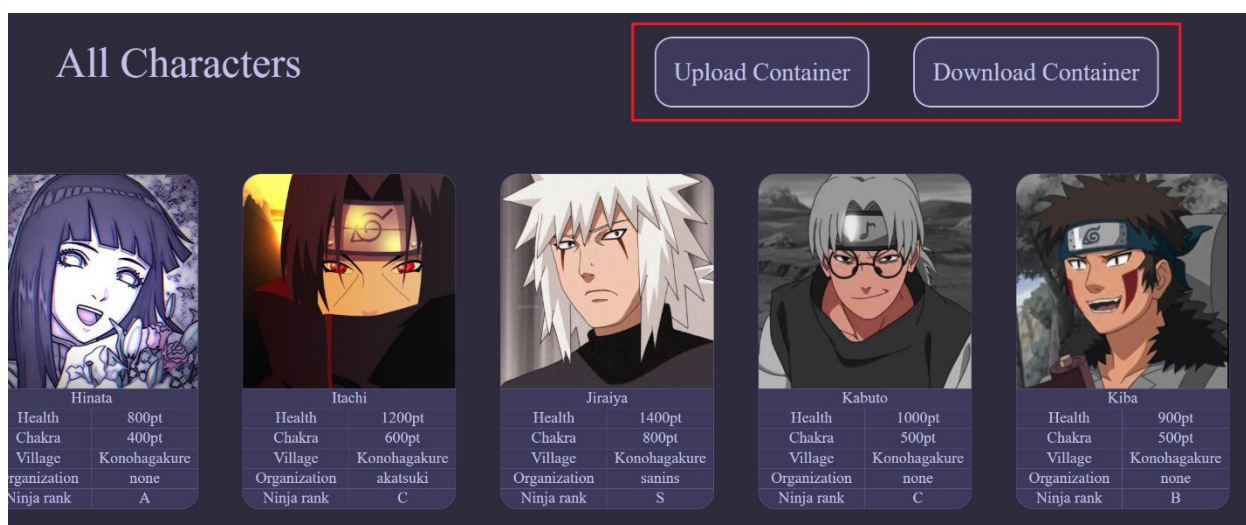


Рисунок 2.5.1 – Внешний вид страницы

3 ТЕСТИРОВАНИЕ

3.1 Открытие диалогового окна

При нажатии на кнопку Upload Container должно открываться диалоговое окно – тест пройден успешно (см. рисунок 3.1.1).

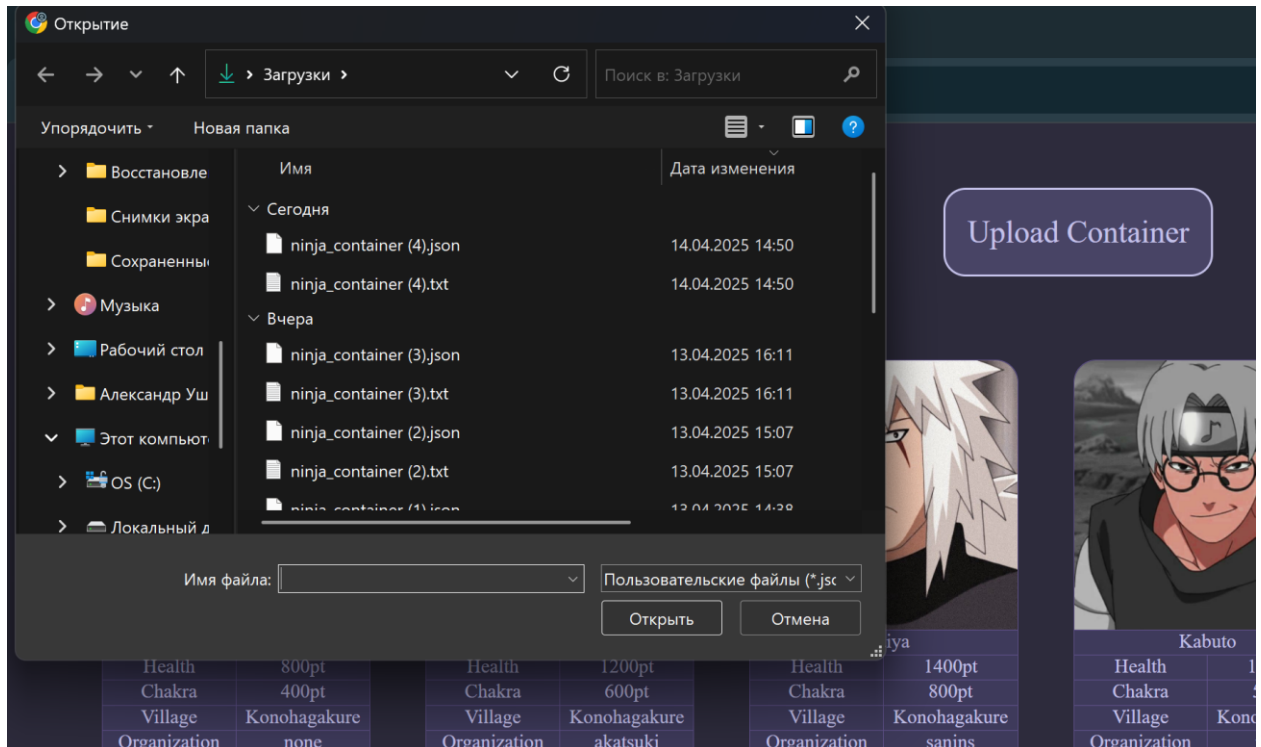


Рисунок 3.1.1 – Диалоговое окно

3.2 Загрузка контейнера на страницу

Изначально последний персонаж контейнера – это Tsunade (см. рисунок 3.2.1).

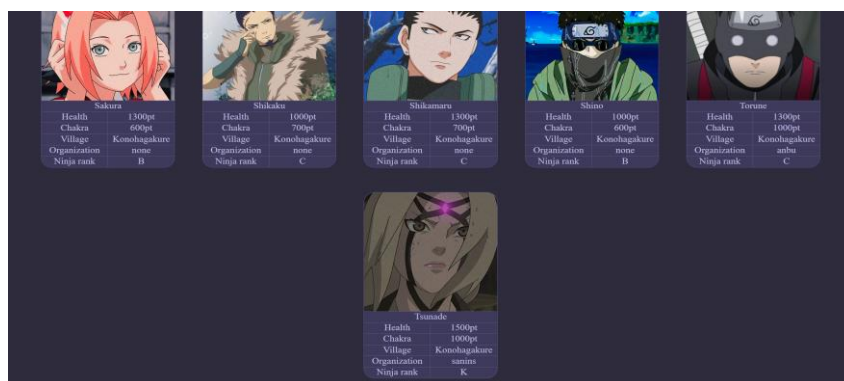


Рисунок 3.2.1 – Исходный контейнер

Загрузим контейнер, в который заранее был добавлен персонаж Гаага. Результаты загрузки из TXT и JSON совпадают, контейнер загружается успешно – тест пройден (см. рисунок 3.2.2).

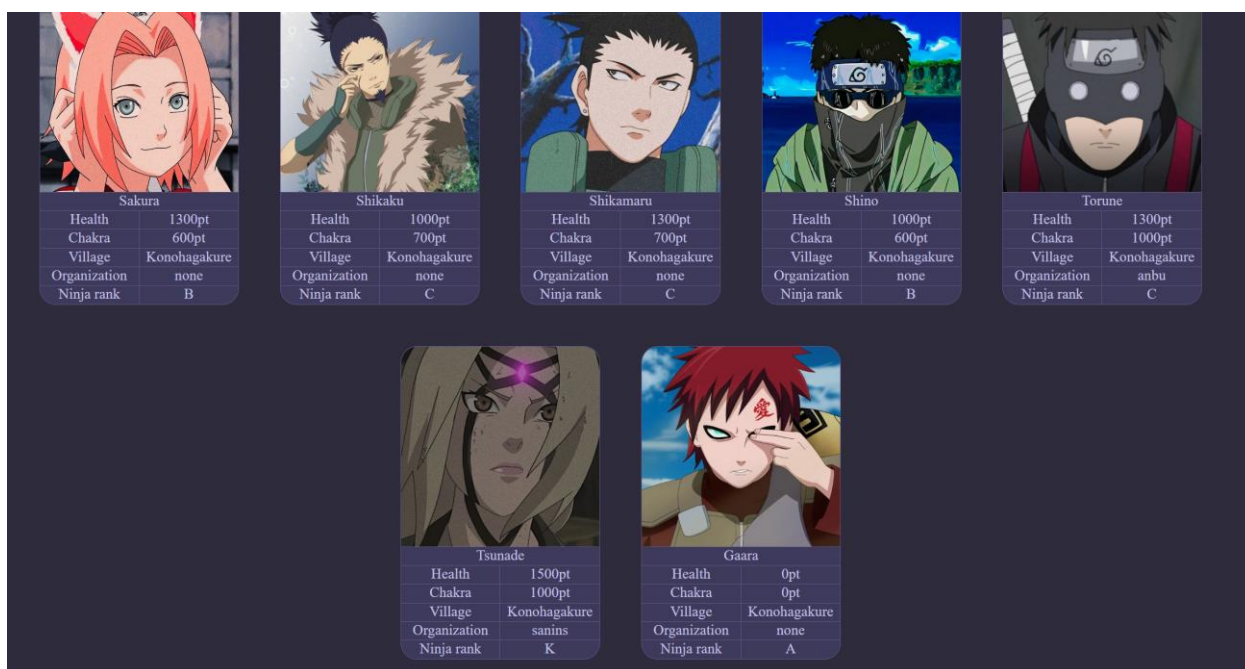


Рисунок 3.2.2 – Успешная загрузка контейнера

3.3 Скачивание контейнера

После нажатия на кнопку Download Container на диск загружаются два файла в форматах JSON и TXT (рис. 3.3.1), представление данных соответствует ожидаемому (рис. 3.3.2 и 3.3.3) – тест пройден успешно.

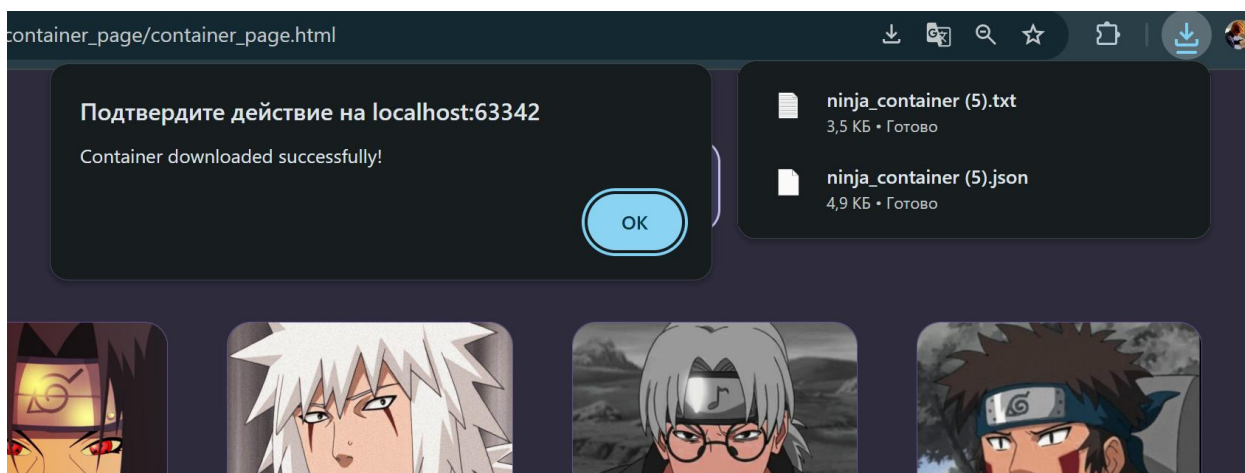


Рисунок 3.3.1 – Скачивание файлов в двух форматах

```
[
  {
    "type": "sensorNinja",
    "data": {
      "name": "Hinata",
      "health": 800,
      "chakra": 400,
      "rank": 1,
      "organization": "none",
      "village": "Konohagakure",
      "appearance": "../../../src/components/characters/hinata/images/hinata_ava.jpg"
    }
  },
  {
    "type": "clairvoyant",
    "data": {
      "name": "Itachi",
      "health": 1200,
      "chakra": 600,
      "rank": 3,
      "organization": "akatsuki",
      "village": "Konohagakure",
      "appearance": "../../../src/components/characters/itachi/images/itachi_ava.jpg"
    }
  }
],
```

Рисунок 3.3.2 – Представление в JSON

```
C: > Users > PC > Downloads > ninja_container (5).txt
1  Type: sensorNinja
2  name: Hinata
3  health: 800
4  chakra: 400
5  rank: 1
6  organization: none
7  village: Konohagakure
8  appearance: ../../../src/components/characters/hinata/images/hinata_ava.jpg
9  arenaView: undefined
10
11 Type: clairvoyant
12 name: Itachi
13 health: 1200
14 chakra: 600
15 rank: 3
16 organization: akatsuki
17 village: Konohagakure
18 appearance: ../../../src/components/characters/itachi/images/itachi_ava.jpg
19 arenaView: undefined
20
21 Type: birdwatchers
22 name: Jiraiya
23 health: 1400
24 chakra: 800
25 rank: 4
26 organization: sanins
27 village: Konohagakure
28 appearance: ../../../src/components/characters/jiraiya/images/jiraiya_ava.jpg
29 arenaView: undefined
```

Рисунок 3.3.3 – Представление в TXT

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы №3 была успешно реализована система сериализации и десериализации данных для проекта «Ninja-Arena». Основной акцент был сделан на обеспечение гибкости и надежности работы с данными, включая их сохранение в локальном хранилище браузера, а также экспорт и импорт в форматах JSON и TXT. Это позволило расширить функциональность проекта, обеспечив пользователям возможность сохранять и загружать данные между сессиями работы, а также обмениваться ими между устройствами.

Ключевым достижением работы стала интеграция механизмов сериализации и десериализации с существующей системой динамического создания объектов. Использование JSON для структурированного хранения данных и TXT для удобного чтения и редактирования вручную продемонстрировало универсальность подхода. Реализованная валидация данных гарантирует их целостность при загрузке, что минимизирует риск ошибок, связанных с некорректными форматами или несоответствием структуре классов.

Особое внимание было уделено оптимизации кода и обеспечению его модульности. Разделение логики на отдельные файлы (например, `prepare_data.js`, `parse_data.js`, `up_download_data.js`) позволило упростить поддержку и дальнейшее расширение функционала. Кроме того, использование принципов объектно-ориентированного программирования, таких как инкапсуляция и полиморфизм, способствовало созданию чистого, читаемого и масштабируемого кода.

Тестирование подтвердило корректность работы всех компонентов системы. Успешно были проверены сценарии загрузки и скачивания данных, а также их отображение на веб-странице. Реализованные механизмы демонстрируют высокую устойчивость к изменениям и готовность к интеграции с дополнительными функциями, такими как облачное хранение или поддержка новых форматов данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] DiscoCode [Электронный ресурс]. – Режим доступа : <https://discocode.ru/content/js/advanced-js/json>.
- [2] JavaScript RU [Электронный ресурс]. – Режим доступа : <https://learn.javascript.ru/localstorage>.